

NETWORK SESSION · 2026.08

“당신이 시켰잖아요”

시키지도 않은 시공비 청구서

믿고 맡긴 AI가 조용히 부풀리는 토큰 건적

출력 토큰을 입력 토큰으로 바꾸는 4 가지 실전 원칙

영어 사고 · KISS/DRY/YAGNI · 설명 최소화 · 웹에서 추론 → 스펙 주입

그리고 캐싱 · 컨텍스트 컴팩션 · 배치 · 라우팅 · 압축까지 — 모든 수치는 재현 가능한 실측입니다 .

토큰은 균등하게 비싸지 않다

2~8×

출력 토큰 / 입력 토큰 단가 비율

1.44×

같은 뜻, 한국어 / 영어 토큰 수
(실측)

0.10×

캐시된 입력 토큰 단가 (Anthropic)

5~15%

에이전트 세션 중 실제 코드 생성 비중

요금은 “토큰 수 × 단가”다. 그런데 단가는 자리마다 다르다.

같은 10,000 토큰이라도 — 캐시된 입력이면 0.3 센트, 생성된 출력이면 15 센트다 (Sonnet 급 기준, 50 배 차이).

따라서 절약의 정답은 “적게 쓰기”가 아니라 “싼 자리로 옮기기”다.

먼저, 요금표를 정확히 읽자

모델 (2026-08 리스트가)	입력 \$/1M	출력 \$/1M	배수
Claude Opus 4.6 / 5	\$5.00	\$25.00	5×
Claude Sonnet 4.6	\$3.00	\$15.00	5×
GPT-5.x (flagship)	\$5.00	\$30.00	6×
Gemini 3.1 Pro	\$2.00	\$12.00	6×
Gemini 2.5 Pro / GPT-5	\$1.25	\$10.00	8×
DeepSeek V4 Flash	\$0.14	\$0.28	2×

왜 출력이 비싼가

입력은 배치로 병렬 처리 (prefill).

출력은 토큰 1 개마다 전체 forward pass
를 순차 실행 (decode).

즉 물리적으로 더 비싸다 — 협상 불가.

핵심 : 추론 (thinking) 토큰도 출력
요금으로 과금된다.

“출력 = 비싼 자리”. 추론 토큰이 출력에 포함된다는 점이 오늘 발표의 전제다.

오늘 가져갈 4 가지 원칙

① 추론은 영어로

한국어 사고 = 같은 의미에 1.4~1.9 배 토큰 .

비싼 출력 자리에서 손해가 발생한다 .

② KISS · DRY · YAGNI

곧 대비 과한 고도화 = 순수 낭비 .

안 쓸 코드의 생성 · 리뷰 · 수정까지 전부 과금 .

③ 장황한 설명 금지

코드 읽으면 아는 걸 3 번 설명 = 출력 토큰 소각 .

verbosity 를 내리면 설명형 출력 40~60% 감소 .

④ 웹에서 추론 → 스펙 주입

긴 추론은 채팅에서 끝내고 , 결론만 넣는다 .

출력 토큰을 입력 토큰으로 환전한다 .

그리고 후반부에 — 캐싱 · 스펙 주도 개발 · 컨텍스트 컴팩션 · 배치 · 압축 · 라우팅까지 6 개 레버를 더 다룬다 .

원칙 ① 추론은 영어로 — 실측치

동일 의미 문장을 tiktoken 으로 직접 인코딩 (사내 검증 스크립트)

문장 (같은 의미)	EN o200k	KO o200k	비율	KO cl100k	비율
로그인 API 추가 지시	19	34	1.79×	52	2.74×
스키마 확인 추론 문장	32	41	1.28×	70	2.19×
함수 동작 설명 문장	20	25	1.25×	33	1.65×
설계 논의 문장	22	28	1.27×	47	2.14×
“에이전트 루프를 다시 짰다”	10	20	2.00×	31	3.10×
에러 분석 문장	18	26	1.44×	36	2.00×
합계 (6 문장)	121	174	1.44×	269	2.22×

해석

최신 토큰라이저 (o200k) 에서도 한국어는 영어 대비 약 1.44 배 . 구형 (cl100k)·Claude 토큰라이저에서는 1.9~2.2 배까지 벌어진다 .

추론 8,000 토큰 기준 : 영어 \$0.120 → 한국어 \$0.173. 요청 1 건당 약 44% 초과 지출 .

실제로 100 번 호출해봤더니 — 결론이 같렸습니다

Gemini API 실패율 100 건 (성공 100/100). 그런데 호출당 비용만 보면 과제에 따라 결론이 뒤집힙니다 .

과제	언어	평균 응답	실제 쓴 글자	호출당 비용	한국어 추가비용
설명형	EN	157 tok	825 자	\$0.001595	—
설명형	KO	203 tok	410 자	\$0.002068	+29.7%
추론형	EN	367 tok	1,040 자	\$0.003744	—
추론형	KO	285 tok	414 자	\$0.002963	-20.9%

“추론형은 한국어가 21 % 싸다”가 아닙니다 . 네 번째 열을 보십시오 — 한국어가 답을 40 % 만 썼습니다 .

같은 분량 (1,000 자) 으로 정규화하면	EN	KO	배수
설명형	\$0.00193	\$0.00504	2.61×
추론형	\$0.00360	\$0.00715	1.99×

사고를 켜고 다시 , 그리고 통계로 못을 박았습니다

앞 장은 사고 (thinking) 꺼진 상태였습니다 . 사고 예산을 켜고 80 건 재측정 — 부호가 갈리는 현상이 똑같이 재현됩니다 .

과제	언어	사고	응답	쓴 글자	호출당 비용	차이
설명형	EN	697	144	822 자	\$0.008441	—
설명형	KO	967	181	366 자	\$0.011515	+36.4%
추론형	EN	1,616	214	570 자	\$0.018374	—
추론형	KO	1,116	210	305 자	\$0.013375	-27.2%

추론형에서 영어가 사고를 1.4 배 더 했습니다 . 한국어가싼 게 아니라 덜 생각하고 덜 쓴 겁니다 .

분량 정규화 (\$/1,000 자)	EN	KO	배수	95% 신뢰구간	p 값
설명형	\$0.0103	\$0.0312	3.03×	[2.68, 3.47]	< 0.001
추론형	\$0.0325	\$0.0442	1.36×	[1.23, 1.51]	6e-10

신뢰구간이 1.0 을 넘지 않습니다 — 분량을 통제하면 한국어가 비싸다는 결론은 예외 없이 유의합니다 .

왜 한국어가 더 비싼가

토크나이저 어휘 배분 문제

o200k 20 만 어휘 중 라틴 문자 포함 토큰 67%,
한글 포함 토큰은 단 1.2% (자체 실측).

" 에이전트 " → 에 / 이 / 전 / 트 = 4 토큰

" agent " → agent = 1 토큰

글자 수는 함정이다

토큰 / 글자 실측 : 영어 0.204 · 한국어 0.526.

한국어는 글자 수가 더 적는데 토큰은 2.6 배 많다 .

512 토큰 청크 = 영어 2,513 자 vs 한국어 973 자 .

RAG 청킹을 글자 수로 자르면 조용히 잘려 나간다 .

실무 규칙

사고 (thinking) 는 영어 → 최종 답변 · 주석 · 커밋 메시지는 한국어 . 비싼 “양”은 영어로 , 사람이 읽을 “질”만 한국어로 .

단, 공짜는 아니다 — 검증된 트레이드오프

영어 사고가 유리한 근거

LLM 의 “reasoning hub” 는 영어 . 비영어 사고를 강제하면 성능이 떨어진다 .

DeepSeek-R1-Distill-Llama-8B: 영어 사고 강제 시 평균 +26.8% .

사용자 언어로 사고를 강제하면 정확도 26% → 17% 하락 (XReasoning) .

즉 영어 사고는 비용 + 품질 양쪽에서 유리한 드문 선택 .

주의할 점

- ① 감시 가능성 저하 — 추론 과정을 한국어 화자가 못 읽는다 .
- ② 한국어 고유 맥락 (호칭 · 존댓말 · 국내 규정) 작업은 예외 .
- ③ 일부 모델은 중국어 사고가 더 효율적 (Qwen3: -40%) .
- ④ 프롬프트에 “영어로 생각해”라고 쓰는 것보다 API 파라미터가 확실하다 .

권고 : 코드 · 수식 · 설계 = 영어 사고 기본값 . 도메인이 한국어 그 자체인 작업만 예외 처리 .

원칙 ② KISS · DRY · YAGNI

KISS

가장 단순한 동작 버전을 먼저 .

추상화 레이어 1 개 = 생성 · 설명 ·
리뷰 · 수정 토큰 4 중 과금 .

“지금 이 골에 필요한 최소 구조”를
프롬프트에 명시 .

DRY

같은 코드 · 같은 로직 · 같은 리워크를
두 번 사지 않는다 .

규칙은 CLAUDE.md / rules 파일
한 곳에 .

단 , 컨텍스트의 DRY 는 처방이
반대다 → 다음 장 .

YAGNI

안 쓸 기능은 만들지 않는다 .

“확장성 고려해서”가 가장 비싼 한마디 .

필요해지면 그때 시킨다 — 그때가 안
오는 경우가 대다수 .

과잉 고도화의 진짜 청구서

토큰은 “생성 시점”에만 드는 게 아니다 . 불필요한 추상화는 이후 모든 세션의 컨텍스트에 영구히 실려 다닌다 .

에이전트 세션에서 실제 코드 생성은 총 토큰의 5~15% . 나머지는 전부 컨텍스트 오버헤드다 .

DRY, 코드에선 “지워라” 컨텍스트에선 “고정하라”

같은 12,000 토큰 프리픽스를 100 번 보내는 4 가지 방법 — 비용이 11 배 갑니다 .

전략	입력 토큰	비용	기준 대비
① 매번 그대로 전송	1,200,000	\$3.60	기준
② 절반으로 요약 · 삭제해서 반복을 줄인다	600,000	\$1.80	-50%
③ 한 글자도 안 바꾸고 그대로 → 캐시 적중	1,200,000	\$0.40	-89%
④ 앞에 타임스탬프 한 줄 → 매번 캐시 미스	1,200,000	\$4.50	+25%

반복을 “줄인” ②보다 “그대로 둔” ③이 4.5 배 싸다

제거할 수 있는 반복은 제거한다 — 코드 · 로직 · 리워크 .

제거할 수 없는 반복은 고정한다 — 시스템 프롬프트 · 규약 · 스키마 .

비싼 것은 반복이 아니라 “변주”다

④는 앞에 동적값 한 줄 붙였을 뿐인데 ①보다 25% 더 낸다 .

캐시는 완전 일치에만 반응한다 — 어중간하게 다른 게 최악 .

“DRY 하면 토큰이 준다” — 5 개로 쪼개서 검증했습니다

#	현장에서 말하는 DRY	판정	실측 근거
A	같은 코드를 다시 쓰지 않는다	참	파일 전체 재출력 vs 변경분만 → - 94%
B	반복되는 컨텍스트를 줄인다	조건부	줄이면 - 50%, 고정하면 - 89%. 처방이 다르다
C	반복되는 리워크를 줄인다	참	6 사이클 (48 턴) 은 1 사이클 (8 턴) 의 16.6 배
D	같은 로직은 한 군데서	참	복붙 3 곳 = 수정 비용 정확히 3 배
E	같은 프롬프트를 최소화한다	조건부	중복 “호출”은 제거 (-30%), 중복 “내용”은 고정

C 가 왜 16.6 배인가 — 리워크는 “한 번 더 시키는 것”이 아니다

턴이 6 배가 되면 비용은 6 배가 아니라 16.6 배가 된다. 매 턴 그때까지의 대화 전체가 입력으로 재전송되기 때문이다. 그래서 DRY 는 코딩 규율이 아니라 요금 정책이다. 사양을 먼저 굳히면 이 곡선 자체를 타지 않는다.

원칙 ③ 장황한 설명을 금지하라

전형적인 낭비 패턴

- ① 코드 쓰기 전에 계획을 산문으로 서술
 - ② 코드 블록
 - ③ 코드가 하는 일을 다시 문단으로 설명
 - ④ 마지막에 “요약” 재서술
- 같은 정보를 3 번 판다. 전부 최고 단가 출력.

처방 — 두 개의 다이얼

verbosity = low → 설명형 출력 40~60% 감소

reasoning_effort: 설계만 high, 구현은 low~medium

- effort 별 월비용 : minimal \$7.92 / medium \$132
- max 는 \$924 — 작업별 배분 시 \$118 (70% 절감)

진단 : reasoning_tokens / output_tokens > 0.8 이면 과다

출력 스타일 (동일 작업)	응답 tok	월 비용 (1,760 콜)	절감
장황 — 계획 + 코드 + 해설 + 요약	342 tok	\$9.03	
보통 — 코드 + 짧은 설명	90 tok	\$2.38	-74%
간결 — 코드 + 3 줄 근거	58 tok	\$1.53	-83%

원칙 ④ 웹에서 추론하고, 결론만 주입하라

출력 토큰을 입력 토큰으로 “환전”하는 것이 이 원칙의 본질이다.

BEFORE — 자연어 한 줄 지시

“로그인 좀 만들어줘”

↓ 모델이 요구사항을 추측 (긴 추론)

↓ 되묻기 / 오해 / 잘못된 방향

↓ 수정 지시 → 재추론 → 재생성

↓ 4~6 회 리워크 루프

입력 2,000 / 출력 12,000 tok

AFTER — 웹에서 추론 → 스펙 주입

1. 웹 채팅에서 설계를 끝까지 논의 (구독 정액)

2. 결론을 스펙 문서로 정리 (엔드포인트 · 스키마 · 에러 · 완료조건)

3. 에이전트는 reasoning_effort 낮춤

4. 스펙 붙여넣고 “그대로 구현” 지시

입력 6,000 / 출력 3,000 tok

입력은 3 배 늘었지만, 총비용은 66% 줄었다. 입력은 싸고, 출력은 비싸기 때문이다.

숫자로 확인 : 66% 절감

Sonnet 급 단가 (\$3 입력 / \$15 출력) 기준, 기능 1 건 구현

시나리오	입력 tok	출력 tok	입력비	출력비	합계
A. 자연어 지시 + 긴 추론 + 리워크	2,000	12,000	\$0.006	\$0.180	\$0.186
B. 웹 추론 → 스펙 주입 + 낮은 effort	6,000	3,000	\$0.018	\$0.045	\$0.063
절감	+200%	-75%			-66%

스케일업하면

개발자 10 명 × 하루 8 건 × 22 일 = 1,760 건 / 월

A 안 \$327/월 → B 안 \$111/월

연간 약 \$2,598 절감 (10 인 팀, 모델 1 종 기준)

주의 : 이건 하한선이다

리워크 1 회당 컨텍스트 전체가 재전송된다 (누적 quadratic).

현장 보고 : 스펙 선행 시 리워크 60~80% 감소,

컨텍스트 전략 병행 시 API 지출 40~70% 감소.

보너스 ⑤ 프롬프트 캐싱 — 입력 자리를 더 싸게

제공사	캐시 읽기 단가	조건
Anthropic	0.10× (90% 할인)	명시적 marker, write 1.25×
OpenAI	0.50× (50% 할인)	자동, 1,024 토큰 이상
Google	0.25× (75% 할인)	implicit / explicit
DeepSeek	0.10× (90% 할인)	자동

실측 시뮬레이션

고정 컨텍스트 20K + 가변 1K,
출력 1.5K, 100 회 호출

캐시 없음 : \$8.55

캐시 적용 : \$3.22

→ 62% 절감 (품질 변화 0)

전제 조건 : 고정 컨텍스트를 프롬프트 “앞쪽”에 두고 순서를 바꾸지 말 것 .

시스템 프롬프트 → 톨 정의 → 레포 맵 → 스펙 → (여기까지 캐시) → 가변 질문 . 앞부분이 1 바이트라도 바뀌면 캐시 전체가 무효화된다 .

실사례 : 캐시 히트율 7% → 84% 개선으로 총 LLM 지출 59~70% 감소 (ProjectDiscovery).

캐싱 심화 — 히트율 계단과 안티패턴

캐시 히트율	100 콜 비용	캐시 미사용 대비
캐시 미사용 (기준)	\$8.55	
0% — 컷는데 매번 미스	\$10.05	+18%
40%	\$7.32	-14%
80%	\$4.59	-46%
95%	\$3.56	-58%
100% (이론 상한)	\$3.22	-62%

→ 리포의 exp07 진단기가 이 동적값을 자동으로 잡아준다.

손익분기 : 0.28 회

캐시 쓰기는 1.25 배 비싸지만, 같은 프리픽스를 한 번만 더 재사용해도 즉시 이득이다.

“재사용이 적어서 안 쓴다”는 거의 항상 오판.

최악의 안티패턴 = 3.1 배

맨 앞에 타임스탬프 · 요청 ID · “현재 시각”을 넣으면 매 호출 캐시가 전부 깨진다. 쓰기 프리미엄만 물어 캐시를 아예 안 쓴 것보다 18% 비싸진다.

캐시 최적 \$3.22 → 안티패턴 \$10.05 = 3.1 배

에이전트 루프 — 비용은 선형이 아니라 2 차 함수다

매 턴 전체 대화가 다시 입력으로 들어간다. 턴이 2 배면 비용은 약 3 배가 된다 (10→20 턴 3.1 배).

세션 길이	누적 비용	증가
5 턴	\$0.26	
10 턴	\$0.74	2.8×
20 턴	\$2.32	3.1×
30 턴	\$4.73	2.0×
50 턴	\$12.09	2.6×

20 턴 세션, 전략별 비용

무대책	\$2.32
캐싱만	\$1.88 (-19%)
캐싱 + 10 턴마다 컴팩션	\$1.31 (-44%)
캐싱 + 5 턴마다 컴팩션	\$0.97 (-58%)

컴팩션 = 오래된 톨 결과 · 중간 로그를 요약으로
교체하는 것. 100 턴 평가에서 84% 절감 보고.

실무 규칙 : 작업이 끝나면 대화를 끊어라 . “ 하나의 대화로 종일 ”이 가장 비싼 습관이다 .

Anthropic: 에이전트는 챗 대비 4 배 , 멀티에이전트는 15 배 토큰을 쓴다 . 에이전트 지출의 99% 이상이 입력 토큰이다 .

스펙 주도 개발 (SDD) 은 조건부로만 참이다

유리한 쪽 — 자체 시뮬레이션

vibe 코딩 40 턴	\$1.75
규약 파일 적용	\$1.09 (-38%)
SDD (스펙 4K + 12 턴)	\$0.51 (-71%)

스펙이 턴 수를 줄이면, 2 차 함수 구간 자체가
잘려 나가기 때문에 절감폭이 크게 나온다.

불리한 쪽 — 외부 반증

Spec-Kit 은 OpenSpec 대비 토큰 +97~109%
(57,740 → 120,947 / 91,729 → 181,040).
무거운 SDD 프레임워크는 오히려 비용을 늘린다.

ETH Zurich (138 이슈, 4 개 모델): LLM 이 생성한
컨텍스트 파일은 성공률 소폭 하락 + 비용 20%↑.

결론 : “스펙을 쓰라”가 아니라 “가볍게, 결정 중심으로 쓰라”

엔드포인트 · 스키마 · 에러 · 완료조건처럼 모델이 추측하면 틀리는 것만 적는다. 코드를 읽으면 아는 것은 스펙에도 쓰지 않는다.

나머지 레버 4종 — 언제 쓰고, 언제 쓰지 말 것인가

배치 API

입력·출력 정액 50% 할인. 캐싱과 중복 적용 가능.

○ 야간 평가·벌크 분류·리포트 생성

× SLA 24 시간. 사용자 대면 채팅·툴 루프 불가.

시맨틱 캐싱

유사 질문을 임베딩으로 매칭해 호출 자체를 제거.

○ FAQ·고객지원·반복 질의 (히트율 20~45%)

× 벤더의 95% 주장은 과장. 임계값 0.8 권장.

모델 라우팅

쉬운 요청은 소형 모델로. RouteLLM 85%↓ @ 품질 95%.

○ 분류·추출·요약 등 난이도 편차 큰 트래픽

× 벤치마크 임계값 그대로 쓰면 과대추정. 재보정 필수.

프롬프트 압축

LLMLingua 최대 20x 압축, 손실 2% 미만.

○ RAG 컨텍스트·장황한 시스템 프롬프트·few-shot

× 코드·수식·짧은 쿼리에는 쓰지 말 것.

레버는 곱해진다 — 스택 적층 시뮬레이션

각 레버는 “앞 단계가 걸러낸 나머지”에 적용된다. 그래서 30% + 50% 는 80% 가 아니라 65% 다.

시작 — 무대책		100%	
① 시맨틱 캐싱 (호출 30% 제거)		70%	-30%p
② 모델 라우팅 (쉬운 건 소형으로)		42%	-28%p
③ 프롬프트 캐싱 (프리픽스 재사용)		21%	-21%p
④ verbosity / effort 조정		16%	-5%p
⑤ 배치 API (비대면 트래픽만)		14%	-2%p

누적 86% 절감 — 업계 보고치 60~80% 의 상한선에 해당한다. 전부 적용했을 때의 이론값으로 읽을 것.

현실적 목표: 상위 2~3 개만 제대로 해도 50~65%. 8 개를 어설프게 하는 것보다 낫다.

무엇부터 할 것인가 — ROI 우선순위

#	레버	절감폭	도입 기간	비고
1	프롬프트 캐싱	-62%	즉시	설정 한 줄 . 손익분기 0.28 회 . 이견 없는 1 순위 .
2	verbosity · effort	-40~60%	즉시	파라미터 한 줄 . 품질 영향은 작업별로 확인 .
3	Batch API	-50%	1 일	비대면 트래픽 한정 . 할인이 정액이라 확실 .
4	스펙 주도 (가볍게)	-38~71%	1 주	습관 변화 필요 . 무거운 프레임워크는 역효과 .
5	컨텍스트 컴팩션	-22~58%	1 주	에이전트 운영 시 필수 . 긴 세션일수록 효과 큼 .
6	모델 라우팅	-49~84%	2~4 주	효과는 최대급이나 임계값 재보정 · 품질 회귀 필요 .
7	시맨틱 캐싱	-20~45%	2~4 주	인프라 추가 . 반복 질의 많은 서비스에서만 .
8	프롬프트 압축	-20~40%	4 주 +	코드 · 수식 금지 . 마지막에 손댈 것 .

절감폭은 각 레버를 “단독 적용”했을 때의 값 . 함께 쓰면 앞 장의 적층 규칙대로 곱해진다 .

오늘부터 적용하는 7 단계 / 버려야 할 7 가지

○ 이렇게 하세요

- 1 사고 언어를 영어로 고정
시스템 프롬프트 / 설정에 명시. 답변만 한국어.
- 2 effort 기본값 low~medium
계획 단계에서만 high.
- 3 verbosity low + 요약 금지
“코드와 3 줄 근거만”
- 4 설계는 웹 채팅에서 끝낸다
정액 요금 구간에서 사고를 소모.
- 5 결론을 스펙 파일로 커밋
specs/*.md — 재사용 가능한 입력 자산.
- 6 고정 컨텍스트 앞쪽 + 캐싱
순서 고정, 앞부분에 동적값 금지.
- 7 주 1 회 토큰 리포트 확인
입력 / 출력 / 추론 / 캐시 히트율 4 개 지표.

× 이걸 버리세요

- × “일단 만들고 아니면 수정하자”
리워크 1 회 = 컨텍스트 전체 재전송.
- × 한국어로 길게 생각하게 두기
같은 사고에 1.4~2.2 배 지출.
- × “확장성 있게, 나중을 대비해서”
YAGNI 위반. 안 쓸 코드가 영구 컨텍스트.
- × 프롬프트 맨 앞에 타임스탬프 · ID
캐시 전멸. 최대 3.1 배 청구.
- × 요약을 요약하고 다시 요약
출력 토큰 3 중 소각.
- × 모든 작업에 최상위 모델 + high
분류 작업에 프런티어 모델은 낭비.
- × 대화 하나로 종일 이어가기
턴이 2 배면 비용은 약 3 배. 2 차 함수.

오늘 숫자는 전부 재현 가능합니다

이 발표의 모든 수치는 슬라이드에 박아 넣은 값이 아니라 , 실행 가능한 스크립트의 출력입니다 .

```
$ pip install -r requirements.txt
$ python run_all.py
$ python run_all.py --model opus
$ python experiments/exp07_analyze_my_prompt.py my.txt
```

단가를 바꿔도 결론이 그대로인가

--model 로 opus / gpt5 / gemini / deepseek 전환 .

배수는 달라져도 “출력 → 입력” 방향은 동일 .

exp01	한 · 영 토크나이저 실측	원칙 ①
exp03	프롬프트 캐싱 히트율	보너스 ⑤
exp05	verbosity · effort	원칙 ③
exp07	내 프롬프트 진단기	실습
exp09	DRY 의 토큰 경제학	원칙 ②

exp02	출력→입력 환전 비용	원칙 ④
exp04	에이전트 루프 · SDD	보너스 ⑥
exp06	배치 · 압축 · 라우팅 · 적층	보너스 ⑦
exp08	Gemini 실시간 과금 측정	실측

+ tools/live_lang_bench.py	— 한 · 영 × 설명형 · 추론형 실호출 벤치	(슬라이드 6 · 7 · N=100/80)
+ tools/stats_test.py	— 부트스트랩 95% CI · Welch · Cliff's δ	· \$/1,000 자 정규화 (슬라이드 7)
+ tools/thinking_sweep.py	— 사고 예산만 바꿔 5 단 스윙	(슬라이드 25 · 240 배 , gemini-25 단가 환산)

특히 exp07 은 여러분의 실제 시스템 프롬프트를 넣으면 캐시 적격성 · 동적값 위치 · 한글 비중 · 중복을 진단해 줍니다 .

같은 정답률 , 240 배 요금 — 30 회 실측

같은 계산 문제 30 회 . 사고 예산만 변경 . 정답 \$38.016 · 오른쪽 열은 6 회 중 정답 횟수 .

사고 예산	n	사고 토큰	응답	비용 / 호출	배수	정답률 (±0.05)
미지정 (기본)	6	0	6	\$0.000155	1.0×	정답 6/6
꿈 (0)	6	0	6	\$0.000158	1.0×	정답 5/6 (1 회 380.16)
512	6	142	6	\$0.001570	10.1×	정답 2/6 ← 품질 붕괴
2048	6	1,114	6	\$0.011302	72.9×	정답 6/6
자동 (-1)	6	3,704	6	\$0.037203	240×	정답 6/6

사고를 끄든 켜든 정답률은 6/6 으로 같습니다 . 요금만 240 배 차이납니다 .

출력 토큰의 99% 가 보이지 않는다

사고가 켜진 구간에서 과금 출력의 99% 가 thoughtsTokenCount 다 (실측 99.6%).

예산 512 는 정답률이 2/6 으로 떨어졌다

어중간한 예산은 추론이 끊겨 품질까지 나빠진다 . 충분히 주거나 아예 끌 것 .

정리

토큰 절약 = 적게 쓰기가 아니라,싼 자리로 옮기기

비싼 자리

출력 토큰 · 추론 토큰 · 한국어 · 리워크

싼 자리

입력 토큰 · 캐시된 입력 · 영어 · 확정된 스펙

한 줄 요약

생각은 밖에서 (웹 · 영어 · 짧게), 실행은 안에서 (스펙 · 낮은 effort · 캐시).

근거 및 검증 자료 (1/2) — 학술 문헌

모든 링크는 원문 기준 . arXiv 번호로 검색하면 바로 나옵니다 .

- Language Model Tokenizers Introduce Unfairness Between Languages**
 토큰화 단계에서 언어 간 불평등 발생 . 일부 언어는 같은 내용에 2.5 배 이상 지불 .
 Petrov, La Malfa, Torr, Bibi (Oxford) · NeurIPS 2023
arxiv.org/abs/2305.15425
- When Models Reason in Your Language: Controlling Thinking Language Comes at ...**
 사용자 언어로 사고를 강제하면 언어일치율 46%→98%, 그러나 정확도 26%→17%.
 XReasoning 벤치마크 · 2025
arxiv.org/abs/2505.22888
- Language Matters: How Do Multilingual Input and Reasoning Paths Affect Large...**
 영어는 reasoning hub. DeepSeek-R1-Distill-Llama-8B 영어 사고 시 평균 +26.8%.
 2025
arxiv.org/abs/2505.17407
- RouteLLM: Learning to Route LLMs with Preference Data**
 강·약 모델 라우팅으로 품질 유지하며 비용 2 배 이상 절감 . 오픈소스 프레임워크 공개 .
 Ong 외 7 인 (UC Berkeley · Anyscale) · ICLR 2025
arxiv.org/abs/2406.18665
- LLMLingua: Compressing Prompts for Accelerated Inference of Large Language M...**
 최대 20× 압축에 성능 손실 1.5 점 . 후속작 LongLLMLingua 는 arxiv.org/abs/2310.06839.
 Jiang, Wu, Lin, Yang, Qiu (Microsoft Research) · EMNLP 2023
arxiv.org/abs/2310.05736
- LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prom...**
 2~5× 압축을 3~6 배 빠르게 . 실무에서 쓰기 좋은 경량 버전 .
 Pan, Wu, Jiang 외 (Microsoft Research) · ACL 2024 Findings
arxiv.org/abs/2403.12968
- GPT Semantic Cache: Reducing LLM Costs and Latency via Semantic Embedding Ca...**
 API 호출 최대 68.8% 감소 , 히트율 61.6~68.8%, positive hit rate 97%+.
 Regmi, Pun · 2024
arxiv.org/abs/2411.05276
- Lost in the Middle: How Language Models Use Long Contexts**
 핵심 정보가 컨텍스트 중간에 있으면 성능 급락 (U 자 곡선) . 앞·뒤에 배치할 것 .
 Liu 외 6 인 (Stanford) · TACL 2024
arxiv.org/abs/2307.03172

근거 및 검증 자료 (2/2) — 벤더 문서 · 자체 실측

token-cost-lab — 본 발표의 자체 실측 코드 (exp01~exp09 + tools/)

tiktoken o200k_base · cl100k_base, 한·영 6 문장쌍, 단가 시뮬레이션

github.com/giyeop-cody/token-cost-lab

Anthropic — Prompt caching / Message Batches API 공식 문서

캐시 읽기 0.1× (90% 할인), 쓰기 1.25×(5분)·2×(1시간), 최소 1,024 토큰. 배치 50% 할인·최대 24 시간·10 만 요청

docs.anthropic.com/en/docs/build-with-claude/prompt-caching

OpenAI — Prompt caching / Batch API 공식 문서

자동 프리픽스 캐싱 (1,024 토큰 이상), 배치 50% 할인·24 시간 SLA·배치당 5 만 요청

platform.openai.com/docs/guides/prompt-caching

Google — Gemini context caching 문서 (implicit / explicit)

캐시 읽기 약 75% 할인. DeepSeek 도 자동 캐싱으로 0.1× 적용

ai.google.dev/gemini-api/docs/caching

Anthropic Engineering — “Effective context engineering for AI agents” (2025.09)

context rot·attention budget 개념, 컴팩션·구조적 노트·서브에이전트 (요약 1,000~2,000 토큰) 전략

anthropic.com/engineering/effective-context-engineering-for-ai-agents

Anthropic Engineering — “How we built our multi-agent research system” (2025)

에이전트는 챗 대비 약 4 배, 멀티에이전트는 약 15 배 토큰 소모

anthropic.com/engineering/multi-agent-research-system

Spec-Kit vs OpenSpec 토큰 벤치마크 (2026) / ETH Zurich 컨텍스트 파일 연구

Spec-Kit 이 OpenSpec 대비 토큰 +97~109%. LLM 생성 컨텍스트 파일은 성공률 소폭 하락 + 비용 20%+ 증가

SDD 반증 근거 — 슬라이드 15

2026 LLM 가격 비교 자료 및 각 벤더 공식 가격 페이지

출력 / 입력 단가 배수 2~8× 확인. 단가는 수시 변동하므로 인용 시 확인 날짜 병기 필요

anthropic.com/pricing · openai.com/api/pricing · ai.google.dev/pricing

질문 받겠습니다

오늘 나온 모든 수치는 리포에서 직접 돌려볼 수 있습니다 .

token-cost-lab — 무엇이 들어 있나

exp01~09 · 원칙별 재현 스크립트 (실패율 없음 , 무료)

tools/ · 라이브 벤치 · 통계 검정 · 사고 예산 스왑

results/ · 실측 원자료 JSONL 110 행 + 검정 출력

demo/ · 페르소나 A/B 토큰 비교 + 메모장 산출물

```
$ git clone https://github.com/giyeop-cody/token-cost-lab
```

```
$ pip install -r requirements.txt && python run_all.py
```



스캔하면 바로 리포로

github.com/giyeop-cody/token-cost-lab

" 우리 모델에도 같은 배수가 나오나요 ? "

exp01 을 자기 모델 토큰라이저로 재실행 . 배수는 다르지만 방향은 같다 .

" 240 배는 과장 아닌가요 ? "

gemini-25 단가 환산 기준 . 실단가로는 149 배 . 원자료 30 행 공개돼 있다 .

" 제일 먼저 뭘 하면 되죠 ? "

캐싱 + verbosity/effort. 설정 한 줄 , 오늘 오후에 켤 수 있다 .